

Trabajo de Recuperación T1

Instrucciones Generales

Este trabajo de recuperación contiene dos proyectos prácticos en un único fichero Markdown:

- RA1: proyecto práctico basado en UT1 (procesos y concurrencia).
- RA2: proyecto práctico basado en UT2 (programación multihilo y sincronización).

No se pide que redactes definiciones largas; el enfoque es 100% práctico. Debes entregar:

- un fichero Markdown con el enunciado y la rúbrica,
 - el código Kotlin/Java desarrollado para cada proyecto (Puedes usar Java o Kotlin),
 - documentación de ejecución y resultados prácticos.
-

RA1: Proyecto práctico de procesos y concurrencia

Exposición RA1

Un **proceso** es un programa en ejecución que posee su propio espacio de memoria independiente. A diferencia de un programa (que es estático hasta que se ejecuta), un proceso es una entidad dinámica que el sistema operativo gestiona y planifica.

La **concurrencia** permite que múltiples procesos se ejecuten de forma simultánea (o aparentemente simultánea en un procesador simple). El sistema operativo multiplexa el tiempo de ejecución para que todos aprovechen al máximo los recursos disponibles.

En **Java/Kotlin**, la clase `ProcessBuilder` nos permite crear y controlar procesos externos desde nuestra aplicación. Podemos capturar su salida, controlar su entrada, y coordinar su ejecución.

La **comunicación entre procesos (IPC)** es crucial: procesos independientes necesitan intercambiar datos. Los mecanismos más comunes son:

- **Tuberías (pipes)**: canal de comunicación unidireccional.
- **Archivos**: procesos escriben y leen datos en archivos compartidos.
- **Sockets**: comunicación por red entre procesos.
- **Redirección de flujos**: dirigir `stdout` de un proceso a `stdin` de otro.

Objetivo

Implementar un mini-proyecto en Kotlin que trabaje con procesos externos, capture su salida y demuestre comunicación entre procesos.

Entregables

- TrabajoRecuperaciónT1-RA1-Kotlin/README.md
- TrabajoRecuperaciónT1-RA1-Kotlin/src/Main.kt
- TrabajoRecuperaciónT1-RA1-Kotlin/RESULTADOS.md
- **Repositorio GitHub público** con el proyecto completo, incluyendo commits que muestren el desarrollo progresivo del código.

Requisitos de entrega RA1

- Crea un repositorio público en GitHub llamado TrabajoRecuperaciónT1-RA1-Kotlin.
- Sube todo el código fuente, documentación y resultados al repositorio.
- Incluye al menos 3 commits que muestren el progreso del desarrollo (por ejemplo: "Implementación inicial de procesos", "Añadida captura de salida", "Comunicación entre procesos").
- En el README del repositorio, incluye una descripción del proyecto, instrucciones de ejecución.
- Entrega el enlace al repositorio junto al documento Markdown.

Enunciado práctico RA1

1. Lanzar procesos externos

- Implementa en Kotlin/Java el lanzamiento de al menos dos procesos externos diferentes usando ProcessBuilder.
- Uno de los procesos debe ser un comando simple que muestre texto en pantalla.
- El otro proceso debe ser otro comando o una aplicación simple (por ejemplo, `cmd /c dir`, `ping`, `notepad`, `calc` o un script propio).

2. Captura de resultados

- Captura y muestra por consola la salida estándar (`stdout`) y la salida de error (`stderr`) de cada proceso.
- Obtén y muestra el código de salida (`exitValue`) de cada proceso.
- Mide el tiempo de ejecución de cada proceso.

3. Comunicación práctica entre procesos

- Implementa un caso en el que un proceso produzca datos y otro proceso los consuma.

- Usa un mecanismo real: redirección de flujos, tubería de proceso o archivo temporal como puente.
- Debe quedar claro qué proceso envía datos, cuál los recibe y cómo se transmiten.

4. Comparación de ejecución

- Ejecuta los mismos comandos de forma secuencial y luego de forma concurrente.
- Mide y registra los tiempos en ambos modos.
- Explica claramente en el documento por qué cambia el tiempo de ejecución.

5. Gestión de errores práctica

- Simula el fallo de un proceso llamando a un comando inexistente o un argumento incorrecto.
- Muestra en consola el mensaje de error y el código de salida.
- Explica brevemente cómo el programa detecta y trata ese fallo.

Ejemplo de estructura

- Main.kt:
 - main() ejecuta las pruebas.
 - lanzarProceso(comando: List<String>): ResultadoProceso.
 - compararSecuencialVsConcurrente().
 - demoComunicacionProceso().
- RESULTADOS.md: ejemplos de salida, tiempos y conclusiones.

Rúbrica de evaluación RA1

La calificación total de RA1 se calcula en una escala de 0 a 10.

Criterio	Peso (%)	Puntos (0-10)
Implementación de procesos externos	30	3,0
Captura de stdout, stderr y códigos de salida	25	2,5
Comunicación entre procesos	20	2,0
Comparación de ejecución secuencial vs concurrente	15	1,5
Documentación de resultados y conclusiones	10	1,0

Puntuación detallada

- **Implementación de procesos externos (3,0 puntos):**
 - 0%: No implementa procesos externos.

- 25%: Implementa parcialmente un proceso externo.
- 50%: Implementa correctamente un proceso externo.
- 75%: Implementa dos procesos externos con algunos errores.
- 100%: Implementa al menos dos procesos externos correctamente con comentarios que explican cada paso y evidencia comprensión de ProcessBuilder.
- **Captura de stdout, stderr y códigos de salida (2,5 puntos):**
 - 0%: No captura salidas ni códigos.
 - 25%: Captura parcialmente alguna salida.
 - 50%: Captura stdout y stderr de un proceso.
 - 75%: Captura salidas y códigos de dos procesos con algunos errores.
 - 100%: Captura y muestra claramente stdout, stderr y códigos de salida de todos los procesos, con documentación en RESULTADOS.md explicando cada salida.
- **Comunicación entre procesos (2,0 puntos):**
 - 0%: No demuestra comunicación.
 - 25%: Intenta comunicación sin éxito.
 - 50%: Demuestra comunicación básica entre dos procesos.
 - 75%: Comunicación funcional con algunos errores.
 - 100%: Demuestra comunicación real con explicación en comentarios y ejemplo documentado.
- **Comparación de ejecución secuencial vs concurrente (1,5 puntos):**
 - 0%: No realiza comparación.
 - 25%: Compara parcialmente sin tiempos.
 - 50%: Realiza comparación con tiempos básicos.
 - 75%: Comparación completa con tabla pero explicación insuficiente.
 - 100%: Comparación documentada con tabla comparativa y explicación clara en RESULTADOS.md.
- **Documentación de resultados y conclusiones (1,0 puntos):**
 - 0%: Sin documentación.
 - 25%: Documentación mínima.
 - 50%: Documentación básica con resultados.
 - 75%: Documentación ordenada con algunas conclusiones.
 - 100%: Documentación completa con análisis personal sobre lo observado.

Nota importante: El alumno deberá exponer y demostrar el ejercicio en una sesión de exposición. El profesor preguntará al alumno sobre el código, decisiones tomadas y resultados obtenidos para verificar la comprensión y autoría del trabajo.

RA2: Proyecto práctico de multihilo y sincronización

Exposición RA2

Un **hilo (thread)** es la unidad más pequeña de ejecución dentro de un proceso. A diferencia de los procesos (que son independientes), los hilos comparten el mismo espacio de memoria.

La **programación multihilo** permite que una aplicación realice múltiples tareas simultáneamente dentro del mismo proceso. Esto es más eficiente que la multiprocesación porque los hilos son ligeros y comparten recursos.

Las **corrutinas en Kotlin** son una forma más avanzada de manejar concurrencia. Permiten escribir código asíncrono de manera secuencial, facilitando el manejo de tareas concurrentes sin los problemas tradicionales de los hilos.

En Kotlin, podemos crear hilos usando `Thread`, `thread{}`, o corrutinas con `launch` y `async` del paquete `kotlinx.coroutines`.

Objetivo

Implementar un mini-proyecto en Kotlin que trabaje con hilos y corrutinas, capture su salida y demuestre comunicación entre ellos.

Objetivo

Implementar un mini-proyecto en Kotlin que use hilos reales, proteja recursos compartidos y resuelva un flujo productor-consumidor.

Entregables

- TrabajoRecuperaciónT1-RA2-Kotlin/README.md
- TrabajoRecuperaciónT1-RA2-Kotlin/src/Main.kt
- TrabajoRecuperaciónT1-RA2-Kotlin/RESULTADOS.md
- **Repositorio GitHub público** con el proyecto completo, incluyendo commits que muestren el desarrollo progresivo del código.

Requisitos de entrega RA2

- Crea un repositorio público en GitHub llamado TrabajoRecuperaciónT1-RA2-Kotlin.
- Sube todo el código fuente, documentación y resultados al repositorio.
- Incluye al menos 3 commits que muestren el progreso del desarrollo (por ejemplo: "Creación de hilos básicos", "Implementación de sincronización", "Productor-consumidor").

- En el README del repositorio, incluye una descripción del proyecto, instrucciones de ejecución y enlace al repositorio de RA1 si corresponde.
- Entrega el enlace al repositorio junto al documento Markdown.

Enunciado práctico RA2

1. Lanzar hilos y corrutinas

- Implementa en Kotlin el lanzamiento de al menos dos hilos/corrutinas diferentes.
- Uno usando la clase Thread o thread{ }.
- Otro usando corrutinas con launch o async.
- Cada uno debe realizar una tarea diferente y mostrar su nombre en consola.

2. Captura de resultados

- Captura y muestra por consola la salida de cada hilo/corrutina.
- Obtén y muestra el resultado o estado de finalización de cada uno.
- Mide el tiempo de ejecución de cada hilo/corrutina.

3. Comunicación práctica entre hilos/corrutinas

- Implementa un caso en el que un hilo/corrutina produzca datos y otro los consuma.
- Usa un mecanismo real: canales (Channels) en corrutinas, o variables compartidas con sincronización básica.
- Debe quedar claro qué hilo/corrutina envía datos, cuál los recibe y cómo se transmiten.

4. Comparación de ejecución

- Ejecuta las mismas tareas de forma secuencial y luego de forma concurrente (usando hilos/corrutinas).
- Mide y registra los tiempos en ambos modos.
- Explica claramente en el documento por qué cambia el tiempo de ejecución.

5. Gestión de errores práctica

- Simula el fallo de un hilo/corrutina lanzando una excepción o cancelando uno.
- Muestra en consola el mensaje de error y el estado de finalización.
- Explica brevemente cómo el programa detecta y trata ese fallo.

Ejemplo de estructura

- Main.kt:
 - main() ejecuta las pruebas.
 - lanzarHilo(comando: () -> Unit): ResultadoHilo.

- `compararSecuencialVsConcurrente()`.
- `demoComunicacionHilos()`.
- `RESULTADOS.md`: ejemplos de salida, tiempos y conclusiones.

Rúbrica de evaluación RA2

La calificación total de RA2 se calcula en una escala de 0 a 10.

Criterio	Peso (%)	Puntos (0-10)
Implementación de hilos y corrutinas	30	3,0
Captura de salidas y estados de finalización	25	2,5
Comunicación entre hilos/corrutinas	20	2,0
Comparación de ejecución secuencial vs concurrente	15	1,5
Documentación de resultados y conclusiones	10	1,0

Puntuación detallada

- **Implementación de hilos y corrutinas (3,0 puntos):**
 - 0%: No implementa hilos ni corrutinas.
 - 25%: Implementa parcialmente un hilo o corrutina.
 - 50%: Implementa correctamente un hilo o corrutina.
 - 75%: Implementa dos hilos/corrutinas con algunos errores.
 - 100%: Implementa al menos dos hilos/corrutinas correctamente con comentarios que explican cada paso y evidencia comprensión de Thread y corrutinas.
- **Captura de salidas y estados de finalización (2,5 puntos):**
 - 0%: No captura salidas ni estados.
 - 25%: Captura parcialmente alguna salida.
 - 50%: Captura salida y estado de un hilo/corrutina.
 - 75%: Captura salidas y estados de dos con algunos errores.
 - 100%: Captura y muestra claramente salidas y estados de todos, con documentación en `RESULTADOS.md` explicando cada uno.
- **Comunicación entre hilos/corrutinas (2,0 puntos):**
 - 0%: No demuestra comunicación.
 - 25%: Intenta comunicación sin éxito.
 - 50%: Demuestra comunicación básica entre dos.
 - 75%: Comunicación funcional con algunos errores.
 - 100%: Demuestra comunicación real con explicación en comentarios y ejemplo documentado.

- **Comparación de ejecución secuencial vs concurrente (1,5 puntos):**
 - 0%: No realiza comparación.
 - 25%: Compara parcialmente sin tiempos.
 - 50%: Realiza comparación con tiempos básicos.
 - 75%: Comparación completa con tabla pero explicación insuficiente.
 - 100%: Comparación documentada con tabla comparativa y explicación clara en RESULTADOS.md.
- **Documentación de resultados y conclusiones (1,0 puntos):**
 - 0%: Sin documentación.
 - 25%: Documentación mínima.
 - 50%: Documentación básica con resultados.
 - 75%: Documentación ordenada con algunas conclusiones.
 - 100%: Documentación completa con análisis personal sobre lo observado.

Nota importante: El alumno deberá exponer y demostrar el ejercicio en una sesión de exposición. El profesor preguntará al alumno sobre el código, decisiones tomadas y resultados obtenidos para verificar la comprensión y autoría del trabajo.

RA5: Auth0 (Autenticación)(Este RA se integra en una aplicación móvil)

Criterios (checks):

- ☐ Se puede iniciar sesión con Auth0 y entrar a la app
- ☐ Al cerrar sesión, se vuelve al login y no se puede entrar sin iniciar sesión
- ☐ Las pantallas "principales" están protegidas (si no hay sesión, manda a login)
- ☐ Se muestra información básica del usuario (por ejemplo: nombre o email)
- ☐ La app no guarda información sensible en texto plano

Escala de calificación (porcentaje de checks cumplidos):

- 100 : Excelente. Auth0 completamente funcional, seguridad bien implementada
 - 75: Bien. Auth0 funcional con login/logout correctos
 - 50 : Aceptable. Auth0 funcional con pequeños problemas
 - 25 : Deficiente. Auth0 básico con limitaciones
 - 0 : Muy deficiente. Auth0 no funcional
-

Entrega final

- Entrega ambos mini-proyectos en carpetas separadas.
- Cada carpeta debe incluir README.md, src/Main.kt y un documento de resultados.
- Comprueba que el código se ejecuta en tu equipo antes de entregar.

- Asegúrate de que el documento Markdown final contiene el enunciado completo y las rúbricas.